

Nama : Daniel Aldy Yupiter

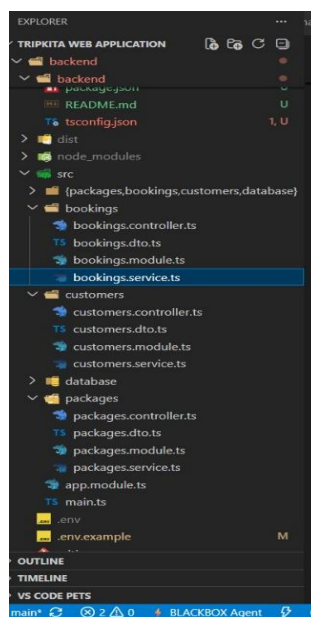
NPM : 23312107

Kelas : IF 23 Fx

DOKUMENTASI Pengerjaan Project

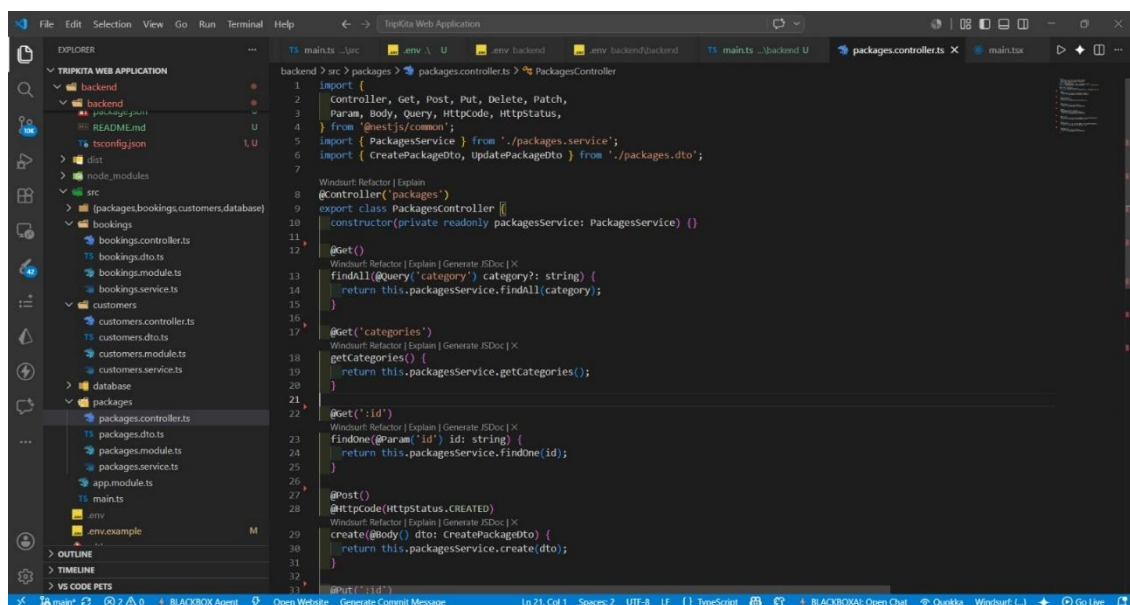
1. Struktur Folder Backend

Struktur backend dibuat menggunakan arsitektur modular untuk mempermudah pengelolaan dan pengembangan sistem. Setiap modul memiliki tanggung jawab masing-masing, yaitu modul Packages untuk mengelola data paket wisata, modul Customers untuk mengelola data pelanggan, modul Bookings untuk mengelola data pemesanan, dan modul Database untuk menangani koneksi ke database PostgreSQL.



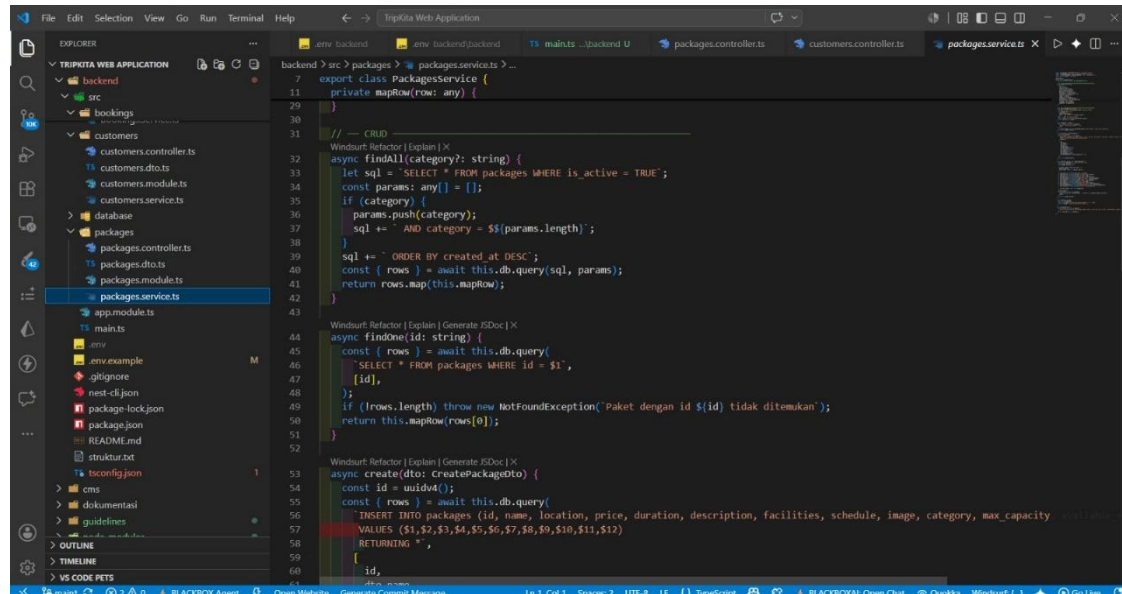
2. Packages Controller

Packages Controller berfungsi menerima request dari frontend terkait data paket wisata. Controller ini mengatur proses pengambilan data, penambahan data, perubahan data, dan penghapusan data paket wisata sebelum diteruskan ke service untuk diproses lebih lanjut.



3. Packages Service

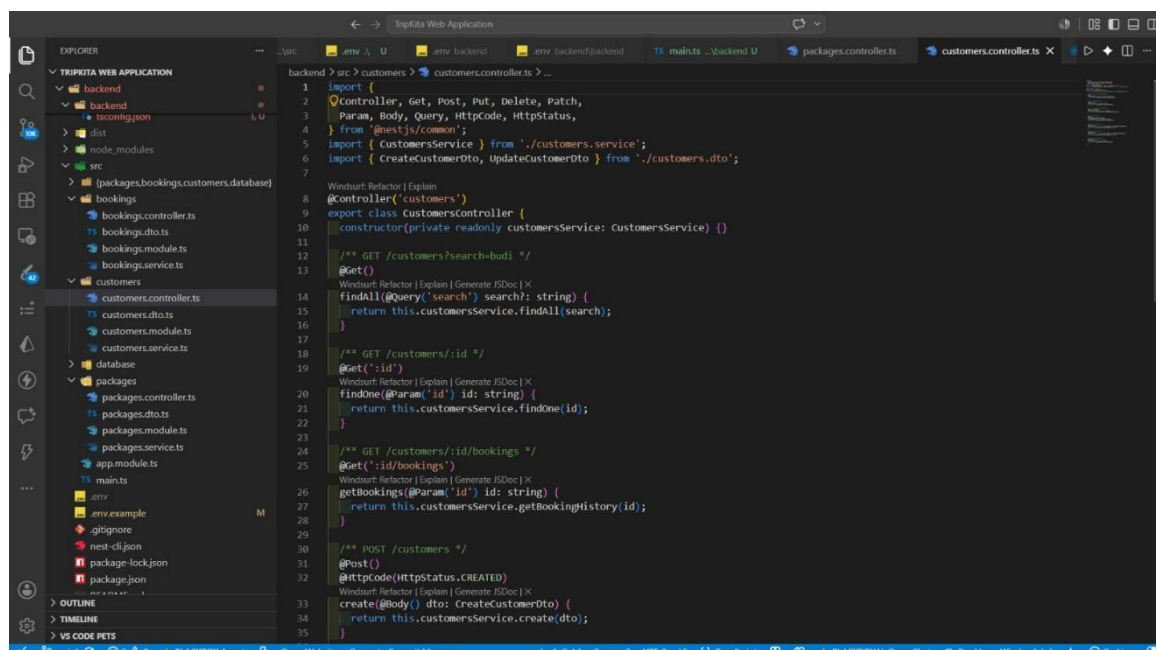
Packages Service berisi logika bisnis yang digunakan untuk mengelola data paket wisata. Service bertugas memproses data yang diterima dari controller, melakukan validasi, serta berinteraksi dengan database untuk menyimpan atau mengambil data paket wisata.



```
7 export class PackagesService {
11 private mapRow(row: any) {
29 }
31 // --- CRUD ---
32 Windurf: Refactor | Explain | X
33 async findAll(category?: string) {
34 let sql = 'SELECT * FROM packages WHERE is_active = TRUE';
35 const params: any[] = [];
36 if (category) {
37 params.push(category);
38 sql += ' AND category = ${params.length}';
39 }
40 sql += ' ORDER BY created_at DESC';
41 const (rows) = await this.db.query(sql, params);
42 return rows.map(this.mapRow);
43 }
44 Windurf: Refactor | Explain | Generate JSDoc | X
45 async findOne(id: string) {
46 const (rows) = await this.db.query(
47 'SELECT * FROM packages WHERE id = ${id}',
48 [id],
49 );
50 if (rows.length) throw new NotFoundException('Paket dengan id ${id} tidak ditemukan');
51 return this.mapRow(rows[0]);
52 }
53 Windurf: Refactor | Explain | Generate JSDoc | X
54 async create(dto: CreatePackageDto) {
55 const id = uuidv4();
56 const (rows) = await this.db.query(
57 'INSERT INTO packages (id, name, location, price, duration, description, facilities, schedule, image, category, max_capacity, is_active)
58 VALUES (${id},${3},${4},${5},${6},${7},${8},${9},${10},${11},${12})
59 RETURNING *',
60 [
61 id,
```

4. Customers Controller

Customers Controller digunakan untuk menangani request yang berkaitan dengan data pelanggan. Modul ini memungkinkan sistem untuk menampilkan data pelanggan, menambahkan pelanggan baru, serta menghubungkan data pelanggan dengan database.



```
1 import {
2 Controller, Get, Post, Put, Delete, Patch,
3 Param, Body, Query, HttpStatusCode, HttpStatus,
4 } from '@nestjs/common';
5 import { CustomersService } from './customers.service';
6 import { CreateCustomerDto, UpdateCustomerDto } from './customers.dto';
7
8 @Controller('customers')
9 export class CustomersController {
10 constructor(private readonly customersService: CustomersService) {}
11
12 /** GET /customers?search=budi */
13 @Get()
14 Windurf: Refactor | Explain | Generate JSDoc | X
15 findAll(@Query('search') search?: string) {
16 return this.customersService.findAll(search);
17 }
18 /** GET /customers/:id */
19 @Get(':id')
20 Windurf: Refactor | Explain | Generate JSDoc | X
21 findOne(@Param('id') id: string) {
22 return this.customersService.findOne(id);
23 }
24 /** GET /customers/:id/bookings */
25 @Get(':id/bookings')
26 Windurf: Refactor | Explain | Generate JSDoc | X
27 getBookings(@Param('id') id: string) {
28 return this.customersService.getBookingHistory(id);
29 }
30 /** POST /customers */
31 @Post()
32 @HttpCode(HttpStatus.CREATED)
33 Windurf: Refactor | Explain | Generate JSDoc | X
34 create(@Body() dto: CreateCustomerDto) {
35 return this.customersService.create(dto);
36 }
```

5. Bookings Controller

Bookings Controller digunakan untuk menangani proses pemesanan paket wisata. Controller menerima data booking yang dikirim dari frontend kemudian meneruskannya ke service untuk disimpan ke dalam database sehingga dapat dikelola oleh administrator.

```
1 import { Controller, Get, Post, Put, Patch, Delete, Param, Body, Query, HttpStatus, } from '@nestjs/common';
2 import { BookingsService } from './bookings.service';
3 import { createBookingDto, UpdateBookingDto } from './bookings.dto';
4
5 @Controller('bookings')
6 export class BookingsController {
7   constructor(private readonly bookingsService: BookingsService) {}
8
9   /** GET /bookings/status=confirmed&paymentStatus=paid */
10  @Get()
11  findAll(
12    @Query('status') status?: string,
13    @Query('paymentStatus') paymentStatus?: string,
14  ) {
15    return this.bookingsService.findAll(status, paymentStatus);
16  }
17
18  /** GET /bookings/stats - dashboard summary */
19  @Get('stats')
20  getStats() {
21    return this.bookingsService.getDashboardStats();
22  }
23
24  /** GET /bookings/:id */
25  @Get('/:id')
26  findOne(@Param('id') id: string) {
27    return this.bookingsService.findOne(id);
28  }
29
30  /** POST /bookings */
31  @Post()
32  create(@Body() createBookingDto: createBookingDto) {
33    return this.bookingsService.create(createBookingDto);
34  }
35}
```

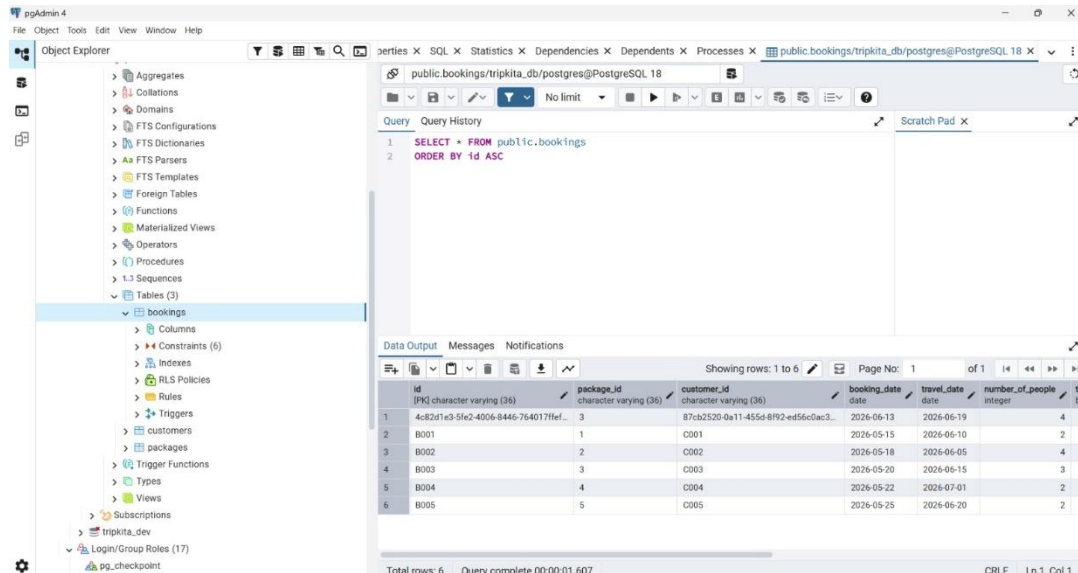
6. Database Connection

Modul Database digunakan untuk menghubungkan aplikasi backend dengan database PostgreSQL. Konfigurasi koneksi meliputi nama database, username, password, serta port yang digunakan sehingga seluruh modul backend dapat mengakses dan mengelola data secara terpusat.

```
1 import { Injectable, OnModuleInit, OnModuleDestroy, Logger } from '@nestjs/common';
2 import { Pool, PoolClient, QueryResult } from 'pg';
3
4 @Injectable()
5 export class DatabaseService implements OnModuleInit, OnModuleDestroy {
6   private pool: Pool;
7   private readonly logger = new Logger(DatabaseService.name);
8
9   onModuleInit() {
10    this.pool = new Pool({
11      host: process.env.DB_HOST || 'localhost',
12      port: parseInt(process.env.DB_PORT || '5432'),
13      user: process.env.DB_USERNAME || 'postgres',
14      password: process.env.DB_PASSWORD || 'password',
15      database: process.env.DB_DATABASE || 'tripkita_db',
16      max: 10,
17      idleTimeoutMillis: 30000,
18      connectionTimeoutMillis: 2000,
19    });
20
21    this.pool.on('error', (err) => {
22      this.logger.error('PostgreSQL pool error:', err);
23    });
24
25    this.logger.log(
26      `Database pool created + ${process.env.DB_HOST || 'localhost'}:${process.env.DB_PORT || '5432'}/${process.env.DB_DATABASE || 'tripkita_db'}`
27    );
28  }
29
30  onModuleDestroy() {
31    this.pool.end();
32    this.logger.log('Database pool closed');
33  }
34}
```

7. Tabel Database

Database TripKita terdiri dari beberapa tabel utama yang digunakan untuk menyimpan data sistem. Tabel Packages menyimpan informasi paket wisata, tabel Customers menyimpan data pelanggan, dan tabel Bookings menyimpan data pemesanan yang dilakukan oleh pelanggan.

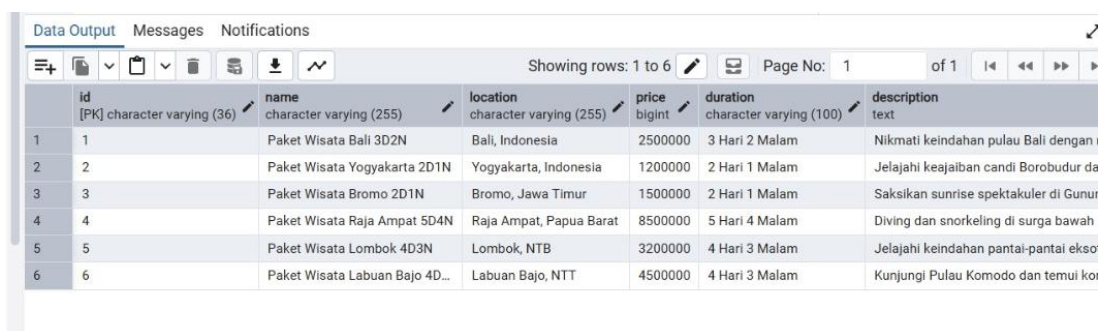


The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including tables like bookings, customers, and packages. The main pane shows a SQL query: `SELECT * FROM public.bookings ORDER BY id ASC`. The Data Output pane displays the results of the query, showing 6 rows of booking data.

id	package_id	customer_id	booking_date	travel_date	number_of_people	total_price
1	4c82d1e3-5fe2-4006-b446-764017f1ef...	87cb2520-9a11-455d-8f92-ed56c0ac3...	2026-06-13	2026-06-19	4	4
2	B001	C001	2026-05-15	2026-06-10	2	2
3	B002	C002	2026-05-18	2026-06-05	4	4
4	B003	C003	2026-05-20	2026-06-15	3	3
5	B004	C004	2026-05-22	2026-07-01	2	2
6	B005	C005	2026-05-25	2026-06-20	2	2

8. Data Booking Tersimpan

Pengujian sistem menunjukkan bahwa data yang diinput melalui form pemesanan pada frontend berhasil diterima oleh backend dan disimpan ke dalam database. Hal ini membuktikan bahwa integrasi antara frontend, backend, dan database telah berjalan dengan baik.



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including tables like packages, customers, and bookings. The main pane shows a SQL query: `SELECT * FROM public.packages ORDER BY id ASC`. The Data Output pane displays the results of the query, showing 6 rows of package data.

id	name	location	price	duration	description
1	Paket Wisata Bali 3D2N	Bali, Indonesia	2500000	3 Hari 2 Malam	Nikmati keindahan pulau Bali dengan...
2	Paket Wisata Yogyakarta 2D1N	Yogyakarta, Indonesia	1200000	2 Hari 1 Malam	Jelajahi keajaiban candi Borobudur da...
3	Paket Wisata Bromo 2D1N	Bromo, Jawa Timur	1500000	2 Hari 1 Malam	Saksikan sunrise spektakuler di Gunung...
4	Paket Wisata Raja Ampat 5D4N	Raja Ampat, Papua Barat	8500000	5 Hari 4 Malam	Diving dan snorkeling di surga bawah...
5	Paket Wisata Lombok 4D3N	Lombok, NTB	3200000	4 Hari 3 Malam	Jelajahi keindahan pantai-pantai ekso...
6	Paket Wisata Labuan Bajo 4D...	Labuan Bajo, NTT	4500000	4 Hari 3 Malam	Kunjungi Pulau Komodo dan temui koi...